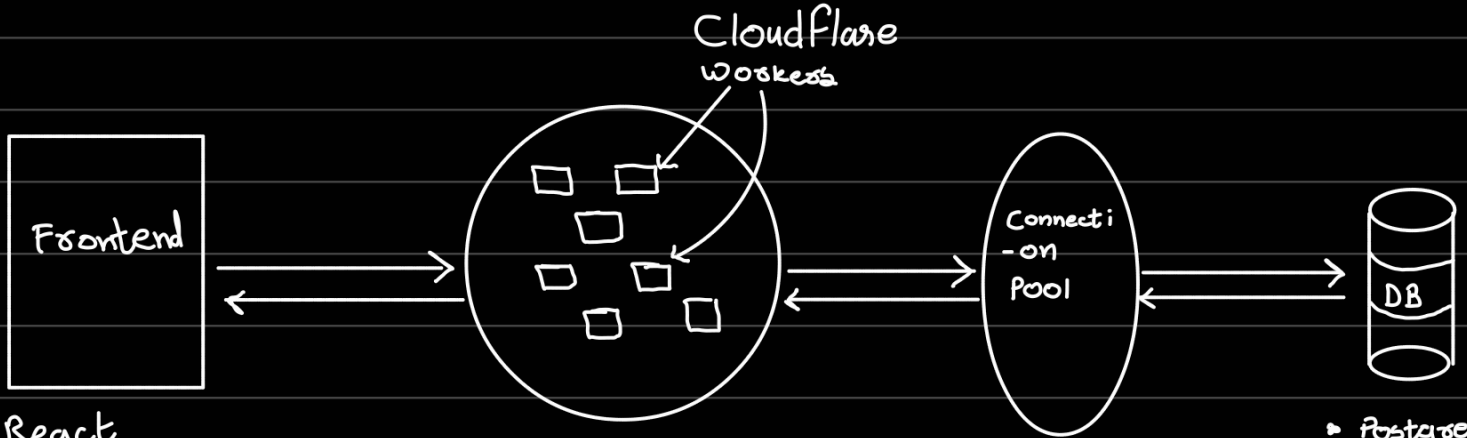
Architecture



- React
- Tailwind
- Recoil

- Server-less backend
- Prisma ORM
- Hono

- prisma accelerate

- Postgress Neon db.

Used Technologies :

- **Server less backend** : Unlike conventional backend servers, NO persistant server here. Upon getting an API request backend goes through Cold start phase.
  - During Cold-start, a random backend worker is spawned which will temporarily act as a backend.
  - If not used for sometime, backend will de-spawn later.
  - Why serverless ?
    Free for a small scale app / pay per use.

- **Hono** : Used to better structure your backend in a serverless env.
  - express too bulky ⟶ increases coldstart time.
  - Hono is lightweight.

```ts
// worker.ts (Cloudflare Worker example without Hono)

export default {
  async fetch(request: Request): Promise<Response> {
    const url = new URL(request.url);

    if (url.pathname === "/") {
      return new Response("Hello, world!");
    }

    if (url.pathname === "/about") {
      return new Response("This is about page");
    }

    return new Response("Not found", { status: 404 });
  },
};
```

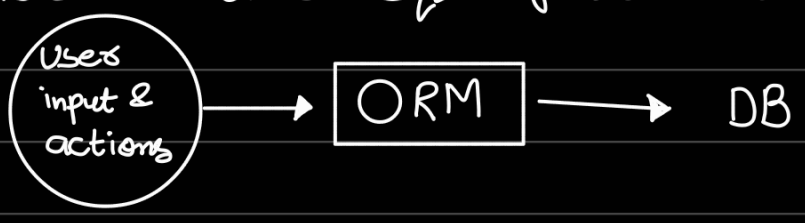
```ts
// worker.ts (Cloudflare Worker with Hono)
import { Hono } from 'hono'

const app = new Hono()

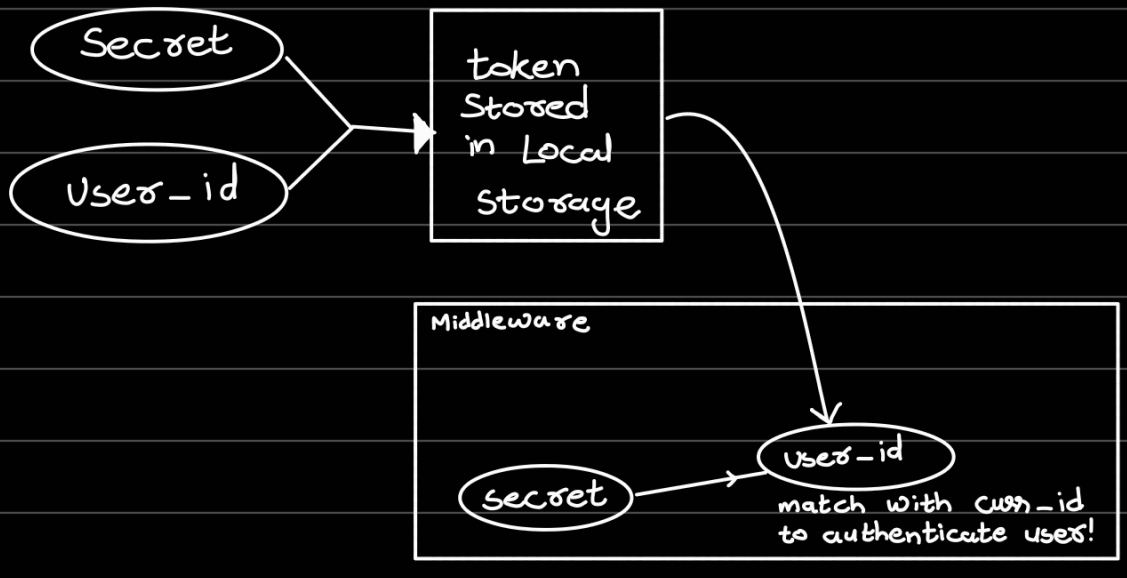
app.get('/', (c) => c.text('Hello, world!'))
app.get('/about', (c) => c.text('This is about page'))

export default app
```

- **Prism ORM** : Object - Relation Mapping.
  - interact with DB by using objects.
  - No need to write SqL queries.
  - Why use ORM ?
    ① easier to switch b/w different DBs. (say Postgres to sqlite)
    ② Reduces risks of sql injection as



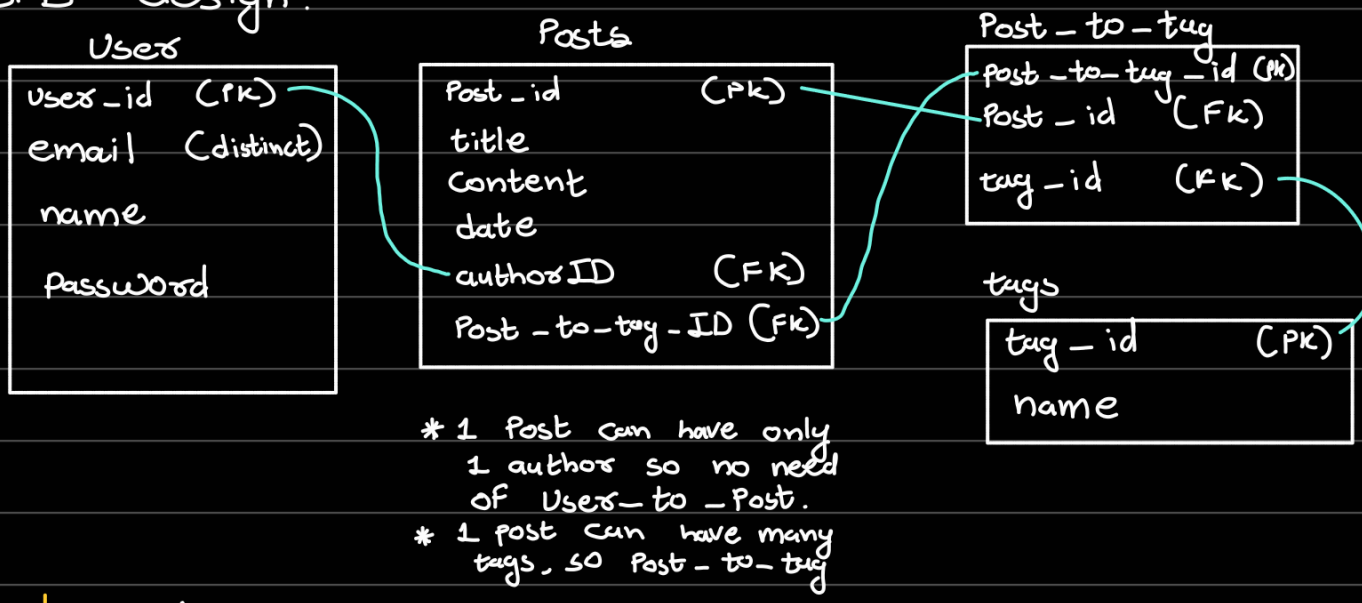
- **JWT** : JSON Web token
  - Used for user authentication & simplified login.
  - Use a secret :



- **Prisma Accelerate** : a Connection pool.
  - for serverless backends, a server can be spawned anywhere.
  - Problems with DB Connection :
    ① A DB has limits to total connections it can make, server-less will exhaust it quickly.
    ② Not safe to expose DB to requests which we don't even know.

- Solution : introduce a centralised connection pool. This pool validates the request , Maintains persistant connection with db and forwards it to db.

- **Neon db** : A postgresdb.
- Sql based.
- RDBMs design.



* 1 Post can have only 1 author so no need of Users_to_Post.
* 1 post can have many tags, so Post_to_tag

- **Zod** : type checking library.

- **Recoil , React , Tailwind** for frontend.

Commonly asked questions :

① What difficulties you faced : (Backend)
→ Backend to DB integration, hence introduced connection POOL.

② Storing styled blogs (frontend)
→ Converting normal blogs to embedded / tagged content so that they can be displayed with style on frontend side.

③ Dockerising (Devops)
→ Dockerising the app to create images and connect all.
→ Connect DB image to a volume to make test db data persistant across all initialisations.

Next Level :

E → Log out button

E → Remove images (because Object stores are expensive.)

M → new style for ./stories with less white space.

E → remove old tags & add new based on pivoted problem statement.

M → loading animations.

M → Guest mode

M → google Oauth

H → Canvas integration on /blog/write /improve write UX

H → Auto switch b/w testing backend & production backend + Scripting + Dockering.

→ Host

Rich Editor logic

① Just Enclose by <Span> tags with Corresponding effect :

hello **world** → hello world

select this and make it bold

- lots of disadvantages / CONS

ⓐ multi Span Creation ; say bold + italics

=> <span for bold> <span for italics> ....

- this can be overcomed by passing content inside.

ⓑ Unsafe during upload !!

when we publish blog we are making axios request with payload = Our blog!

So a user can use postman with artificial payload like :

<Span className: translate: 5000px) _

This breaks frontend !!

⑥ List of possible effects + index range.

maintain a global :

{

bold : {0, 10} {15, 20},

italics :

}

this is the style sheet !!

Cons

① indexing is a mess ! say :

Bold

hello world

01 234 5 6 78910

Bold : {0, 4}

we enter space b/w or paste content the normalizing the global styles with new content is difficult.

© Tree based structure

- Optimal + mostlikely used by Google Doc + Notion.

Create a node for typing and split the node on effects



if DOM → Tree ⇒ slow

take all control from DOM, make changes in Tree directly and render later!